

Detailed Analysis of Codes in 1D CTMC

Project Overview

Project Structure

Data Flow Analysis

Input Data Format

Data Preprocessing Procedure

Calculation Process

Output Procedure

Core Architecture

Configuration System (commonConfig.py)

Default Model Parameters

CTMC Numerical Parameters

Options Parameters

Optimizer Parameters

Data Filtering Criteria

State Space Expansion Parameters

Log Configuration

Detailed Analysis of Each Module

Module 1: src/ctmc_model.py

1.1 Unified Entry Function

1.2 Construction of the Q matrix in the CEV model

1.3 Construction of the Q matrix in the CEV-DEJD model

1.4 Construction of the Q matrix in the CEV-RS model

1.5 Q Matrix Validation

Module 2: src/option_pricing.py

2.1 Unified Entry Function

2.2 Standard Option Pricing (CEV and CEV-DEJD)

2.3 CEV-RS Option Pricing

2.4 Numerical Stability of the Transition Probability Matrix

Module 3: src/parameter_estimation.py

3.1 Parameter Estimation Process

3.2 Calculating Pricing Performance

3.3 Parameter Validation

3.4 Regime Analysis Tools (CEV-RS)

Module 4: utility/data_processor.py

4.1 Data Discretization

4.2 Setting the State Space Range

4.3 Filtering Option Data

4.4 Data Analysis Tools

4.5 Parameter Suggestion Tool

Module 5: utility/file_utils.py

5.1 Read Data

5.2 Save Results

5.3 Backup and Cleanup

5.4 Load Historical Results

Operation Process

Project Overview

This project has developed a futures option pricing system based on the approximation method of one-dimensional continuous-time Markov chains (CTMC), specifically for pricing put options on gold futures (AU2412).

Three models in this project:

- **CEV Model** (Constant Elasticity of Variance)
- **CEV-DEJD Model** (CEV with Double Exponential Jump Diffusion)
- **CEV-RS Model** (CEV with Regime Switching)

Project Structure

```
一维CEV/
├─ main.py                # Entry point of the main program
├─ commonConfig.py        # Global Configuration Center
├─ src/
│   ├─ ctmc_model.py      # Core implementation of the CTMC model
│   │   (construction of the Q matrix)
│   ├─ parameter_estimation.py  # Parameter estimation and calibration
│   └─ option_pricing.py    # Option pricing algorithm (dynamic
programming)
├─ utility/
│   └─ data_processor.py    # Data processing and state space
configuration
│   └─ file_utils.py
│   └─ visualization.py    # Visualization tools (only the early
termination boundary of CEV-RS)
├─ input/
│   └─ Futures_au2412.csv   # Futures price data (date, closing
price)
│   └─ Futures_put_options_au2412.csv # Options market data
├─ result/                # Output Results
│   └─ batch_pricing_report_*.csv
├─ log/                   # Operation Log
│   └─ run_*.log
```

Data Flow Analysis

Input Data Format

Futures Data (Futures_au2412.csv):

```
Date      close
20240306,503.24
20240307,508.3
...
```

Options Data (Futures_put_options_au2412.csv):

```
Date,market_price,F_t,K,volume,Rf,MONEYNESS,TAU
20240603,1.08,555.22,464,129,0.01953,0.835704766,0.479452055
20240603,6.88,555.22,520,126,0.01953,0.936565686,0.479452055
...
```

Data Preprocessing Procedure

```
Original Data
↓
[read_futures_data()] Read Futures Price
↓
[read_option_data()] Read Futures Options Data
↓
[filter_option_data()] Apply Liquidity Filtration
↓
[set_state_space_range()] Set the discretization range based on model type and
characteristics of options
↓
[discretize_data()] Construct Equivalent Grids
```

Calculation Process

```
filtered_data
↓
[estimate_parameters_from_options()] Parameter Calibration
├─ Reduce Calculation Complexity: num_bins=100, N_t=100
├─ Goal: Minimization of relative logarithmic error
├─ Optimization Algorithm: L-BFGS-B (CEV/CEV-DEJD) or Multiple Starting
Points (CEV-RS)
└─ Output: optimal_params (numpy array)
↓
[calculate_daily_pricing_performance()] Use the optimal parameters for pricing
├─ num_bins=500, N_t=1000 (Final Pricing Results)
├─ Calculate the price of each option pricing model
└─  $RAE = |P_{model} - P_{market}| / P_{market}$ 
```

Output Procedure

```
results_dataframe
↓
[save_results()] Save as CSV
├─ batch_pricing_report_{model_type}_{timestamp}.csv
└─ Format: Date, K, TAU, Mkt_Price, Model_Price, RAE_%
```

Core Architecture

Configuration System (commonConfig.py)

Function: Global configuration center, which defines all model parameters and running parameters

Default Model Parameters

```
DEFAULT_MODEL_PARAMS = {
    # CEV Basic Parameters
    'sigma': 0.2,          # Volatility Scaling
    'beta': -0.5,          # Elasticity coefficient (negative value indicates
                           # reverse leverage)

    # CEV-DEJD Extended Parameters
    'lambda': 0.1,         # Poisson Process Intensity (jump frequency)
    'p': 0.5,              # Probability of Jumping Upwards
    'eta1': 2.0,           # Upward Jump Attenuation Rate (controls the size of
                           # upward jumps)
    'eta2': 2.0,           # Downward Jump Attenuation Rate (controls the size
                           # of downward jumps)

    # CEV-RS Extended Parameters
    'sigma1': 0.16,        # State 1 Volatility
    'beta1': 1.0,          # State 1 Elasticity (positive value = reverse
                           # leverage, calm market)
    'sigma2': 0.16,        # State 2 Volatility
    'beta2': -1.0,         # State 2 Elasticity (Negative value = Normal
                           # Leverage, Pressured Market)
    'lambda12': 1.0,       # Transition rate from state 1 to state 2
    'lambda21': 7.0,       # Transition rate from state 2 to state 1
    'initial_regime': 1     # Initial Regime (1 or 2)
}
```

CTMC Numerical Parameters

```
DEFAULT_CTMC_PARAMS = {
    'N': None,             # Number of futures observations (calculated from
                           # the data)
    'N_t': 1000,           # Number of time steps (final pricing)
    'N_s': 500,            # Number of cells in the state space (final pricing)
    'dt': None,            # Time Step (= toT/N_t)
    'x_lower': None,       # Lower Bound of State Space
    'x_upper': None,       # Upper Bound of State Space
    'num_bins': 500,       # Discretized Grid Number
    'F': None              # Array of Futures Prices
}
```

Options Parameters

```

DEFAULT_OPTION_PARAMS = {
    'r_f': None,          # risk-free rate
    'K': None,            # strike price
    'toT': None,          # time to maturity(year)
    'option_type': 'put', # 'call' or 'put'
    'option_am_eu': 'am', # 'am' (American Type) 或 'eu' (European Type)
}

```

Optimizer Parameters

Independent optimization configuration for each model:

```

OPTIMIZER_PARAMS = {
    'cev': {
        'bounds': [(0.01, 0.99), (-10.0, 10.0)], # (sigma, beta)
        'tol': 1e-8,
        'method': 'L-BFGS-B'
    },
    'cev_dejd': {
        'bounds': [
            (0.01, 0.99), # sigma
            (-10.0, 10.0), # beta
            (0.01, 2.0), # lambda
            (0.01, 0.99), # p
            (1.1, 10.0), # eta1 (>1 for MGF)
            (0.1, 10.0) # eta2
        ],
        'tol': 1e-8
    },
    'cev_rs': {
        'bounds': [
            (0.05, 0.99), # sigma
            (-10.0, 10.0), # beta1
            (0.05, 0.99), # sigma2
            (-10.0, 10.0), # beta2
            (0.01, 20.0), # lambda12
            (0.01, 20.0) # lambda21
        ],
        'tol': 1e-8
    }
}

```

Data Filtering Criteria

```

OPTION_FILTER_CRITERIA = {
    'tau_min': 15 / 365, # Minimum Expiration Time: 15 days
    'tau_max': 180 / 365, # Maximum Expiration Time: 180 days
    'moneyness_min': 0.81, # Minimum Real Value Degree: 0.81 (deep real
value)
    'moneyness_max': 1.02, # Maximum Real Value Degree: 1.02 (mildly
positive value)
    'volume_min': 100, # Minimum Trading Volume: 100
    'price_min': 0.50 # Minimum Market Price: 0.50
}

```

State Space Expansion Parameters

Select the expansion ratio based on the model type:

```
STATE_SPACE_PARAMS = {
    'cev': {'delta_upper': 0.05, 'delta_lower': 0.02},
    'cev_dejd': {'delta_upper': 0.15, 'delta_lower': 0.10},
    'cev_rs': {'delta_upper': 0.10, 'delta_lower': 0.08},
}
```

Log Configuration

```
LOG_CONFIG = {
    'level': 'DEBUG',
    'format': '<green>{time:YYYY-MM-DD HH:mm:ss.SSS}</green> | <level>{level}</level> | <cyan>{message}</cyan>',
    'rotation': '10 MB'
}
```

Detailed Analysis of Each Module

Module 1: src/ctmc_model.py

Function: Constructs the generating matrix (\$Q\$ matrix), which is the mathematical core of the CTMC approximation.

1.1 Unified Entry Function

```
def construct_theoretical_q_matrix(grid, initial_price, params,
model_type='cev'):
    """
    Parameters:
        grid: Price state grid center points
        initial_price: F_0 used for local volatility scaling (x/F_0)^β
        params: Parameter vector
            CEV: [sigma, beta]
            CEV-DEJD: [sigma, beta, lambda, p, eta1, eta2]
            CEV-RS: [sigma1, beta1, sigma2, beta2, lambda12, lambda21]
    Returns: Q matrix (N×N for CEV/CEV-DEJD, 2N×2N for CEV-RS)
    """
```

1.2 Construction of the Q matrix in the CEV model

Mathematical Formula (Paper Eq. 4):

For the price grid $\{x_1, \dots, x_N\}$, the elements of the generator matrix Q are:

- Downward transition: $q_{i,i-1} = \max\left[\frac{D(x_i)}{2h^2} - \frac{\mu(x_i)}{2h}, 0\right]$
- Upward transition: $q_{i,i+1} = \max\left[\frac{D(x_i)}{2h^2} + \frac{\mu(x_i)}{2h}, 0\right]$
- Diagonal (negative value): $q_{i,i} = -\sum_{j \neq i} q_{i,j}$

Where:

- Grid step size: $h = x_{i+1} - x_i$
- Diffusion coefficient: $D(x) = \left[\sigma(x/F_0)^\beta \cdot x \right]^2$
- Drift term: $\mu(x) = 0$ (The CEV model has no drift term under the futures risk-neutral condition)

Key Code Implementation:

```
def construct_cev_q_matrix(grid, initial_price, params):
    sigma, beta = params
    h = grid[1] - grid[0]
    n_states = len(grid)
    q_matrix = np.zeros((n_states, n_states))

    for i, x in enumerate(grid):
        # Calculate local volatility
        volatility_term = sigma * (x / initial_price) ** beta
        diffusion_sq = (volatility_term * x) ** 2
        drift = 0.0

        if i > 0:
            q_matrix[i, i-1] = max(diffusion_sq / (2 * h**2) - drift / (2 * h),
0)

        if i < n_states - 1:
            q_matrix[i, i+1] = max(diffusion_sq / (2 * h**2) + drift / (2 * h),
0)

        # Ensure the row sum equals 0 on the diagonal
        q_matrix[i, i] = -np.sum(q_matrix[i])

    return q_matrix
```

1.3 Construction of the Q matrix in the CEV-DEJD model

Mathematical Formula (Paper Eq. 13-14):

$$Q = \Lambda_D + \Lambda_J$$

- **Λ_D (Diffusion part with jump-corrected drift):** The drift includes the jump compensator: $\mu(x) = -\lambda \zeta x$, ensuring the risk-neutral condition $dF/F = 0$.
- **Λ_J (Jump part, integrated from the Lévy measure):**

The elements of the jump part are obtained by integrating the Lévy measure:

$$(\Lambda_J)_{ij} = \int_{\alpha_i(j-1)}^{\alpha_i(j)} \nu(x_i, dy) dy$$

where the Lévy measure is:

$$\nu(x, dy) = (x/S_0)^\beta \lambda \left[p \eta_1 (y+1)^{-\eta_1-1} \mathbf{1}_{y>0} + (1-p) \eta_2 (y+1)^{\eta_2-1} \mathbf{1}_{-1<y<0} \right] dy$$

Definition of $\alpha_i(j)$ (Paper Eq. 13):

$$\alpha_i(j) = \frac{1}{2} \left(\frac{x_j}{x_i} - 1 \right) + \frac{1}{2} \left(\frac{x_{j+1}}{x_i} - 1 \right)$$

Key Code Implementation:

```

def construct_cev_dejd_q_matrix(grid, initial_price, params):
    sigma, beta, lambda_jump, p, eta1, eta2 = params

    # Calculate jump compensator  $\zeta$  (ensuring risk-neutrality)
    zeta = calculate_zeta(p, eta1, eta2)

    # Construct the diffusion part  $\Lambda_D$ 
    lambda_d = construct_diffusion_matrix(
        grid, initial_price, sigma, beta, lambda_jump, zeta
    )

    # Construct the jump part  $\Lambda_J$ 
    lambda_j = construct_jump_matrix(
        grid, initial_price, lambda_jump, p, eta1, eta2, beta
    )

    #  $Q = \Lambda_D + \Lambda_J$ 
    q_matrix = lambda_d + lambda_j
    return q_matrix

```

Jump Compensator Calculation (Eq. 11):

```

def calculate_zeta(p, eta1, eta2):
    """
     $\zeta = E[e^{\Lambda_Y}] - 1 = p \cdot \eta_1 / (\eta_1 - 1) + (1-p) \cdot \eta_2 / (\eta_2 + 1) - 1$ 
    Requirement:  $\eta_1 > 1$  (condition for the existence of MGF for upward jumps)
    """
    if eta1 <= 1:
        logger.warning(f" $\eta_1 = \{eta1\} \leq 1$  - MGF does not exist; clamping to 1.01")
        eta1 = 1.01
    zeta = p * eta1 / (eta1 - 1) + (1 - p) * eta2 / (eta2 + 1) - 1
    return zeta

```

Three Cases for Jump Matrix Construction:

- **Pure upward jump** ($\alpha_{j-1} > 0$ and $\alpha_j > 0$):

```

lambda_j[i, j] = common_factor * p * (
    (alpha_jm1 + 1) ** (-eta1) - (alpha_j + 1) ** (-eta1)
)

```

- **Pure downward jump** ($\alpha_{j-1} < 0$ and $\alpha_j < 0$):

```

lambda_j[i, j] = common_factor * (1 - p) * (
    (alpha_j + 1) ** eta2 - (alpha_jm1 + 1) ** eta2
)

```

- **Mixed case** ($\alpha_{j-1} < 0 \leq \alpha_j$, jump crosses the $y=0$ line):

```

down_part = (1 - p) * (1.0 - (alpha_jm1 + 1) ** eta2)
up_part = p * (1.0 - (alpha_j + 1) ** (-eta1))
lambda_j[i, j] = common_factor * (down_part + up_part)

```


Boundary State Handling::

```
for i in range(n_states):  
    if i == 0 or i == n_states - 1:  
        continue
```

1.4 Construction of the \$Q\$ matrix in the CEV-RS model

Mathematical Formula (Paper Eq. 18):

The $2N \times 2N$ block generator matrix is:

$$Q = \begin{pmatrix} \Lambda_1 - \lambda_{12}I & \lambda_{12}I \\ \lambda_{21}I & \Lambda_2 - \lambda_{21}I \end{pmatrix}$$

State Space Layout:

- States 0 to $N-1$: Price states in Regime 1
- States N to $2N-1$: Price states in Regime 2

Key Code Implementation::

```
def construct_cev_rs_q_matrix(grid, initial_price, params):  
    sigma1, beta1, sigma2, beta2, lambda12, lambda21 = params  
    n_states = len(grid)  
  
    # Construct two independent CEV matrices  
    lambda1 = construct_cev_q_matrix(grid, initial_price, [sigma1, beta1])  
    lambda2 = construct_cev_q_matrix(grid, initial_price, [sigma2, beta2])  
    I_n = np.eye(n_states)  
  
    # Assemble the 2Nx2N block matrix  
    q_matrix = np.block([  
        [-lambda12 * I_n + lambda1, lambda12 * I_n],  
        [lambda21 * I_n, -lambda21 * I_n + lambda2],  
    ])  
  
    return q_matrix
```

Economic Interpretation:

- $\lambda_{12} = 1.0$: Regime 1 lasts for an average of 1 year.
- $\lambda_{21} = 7.0$: Regime 2 lasts for an average of 0.14 years (about 51 days).
- $\beta_1 = 1.0 > 0$: Inverse leverage, characteristic of a calm market.
- $\beta_2 = -1.0 < 0$: Normal leverage, characteristic of a stressed market.

1.5 \$Q\$ Matrix Validation

```
def validate_q_matrix(q_matrix, model_type='cev'):  
    """  
    Check if the Q matrix satisfies the generator matrix conditions:  
    1. Diagonal elements  $\leq 0$   
    2. Off-diagonal elements  $\geq 0$   
    3. Sum of each row equals 0  
    """
```

```

"""
diagonal_valid      = np.all(np.diag(q_matrix) <= 0)
off_diag            = q_matrix - np.diag(np.diag(q_matrix))
off_diagonal_valid  = np.all(off_diag >= -1e-10)    # Small tolerance
row_sums            = np.abs(q_matrix.sum(axis=1))
row_sum_valid       = np.all(row_sums < 1e-8)

# Special check for CEV-RS: dimension must be even
if model_type == 'cev_rs' and q_matrix.shape[0] % 2 != 0:
    is_valid = False

return diagonal_valid and off_diagonal_valid and row_sum_valid

```

Module 2: src/option_pricing.py

Functionality: Option pricing using dynamic programming methods.

2.1 Unified Entry Function

```

def price_option(q_matrix, bins_centers, model_params, option_params,
ctmc_params, model_type):
    """
    Parameters:
        q_matrix: Transition rate matrix
        bins_centers: Bin centers
        model_params: Dictionary of model parameters
        option_params: Dictionary of option parameters
        ctmc_params: Dictionary of CTMC parameters
        model_type: 'cev' | 'cev_dejd' | 'cev_rs'
    Returns:
        CEV/CEV-DEJD: (Option price, Value matrix)
        CEV-RS: (Option price, Value matrix, Early exercise boundary results)
    """

```

2.2 Standard Option Pricing (CEV and CEV-DEJD)

Mathematical Formula (Paper Eq. 5):

Backward recursion:

$$V(t_i, s) = \max \left\{ h(s), e^{-r \cdot dt} \cdot \sum_j P_{ij} \cdot V(t_{i+1}, s_j) \right\}$$

Step 1: Construct the Transition Probability Matrix

```

# P = exp(Q*dt)
p_matrix = expm(q_matrix * time_step)

# Clip and normalize to ensure probability validity
p_matrix = np.clip(p_matrix, 0, 1)
row_sums = p_matrix.sum(axis=1)
p_matrix = p_matrix / row_sums[:, np.newaxis]

```

Step 2: Initialize the Value Matrix

```

n_time_steps = ctmc_params['N_t']
n_states = ctmc_params['N_s']
value_matrix = np.zeros((n_states, n_time_steps + 1))

# Terminal condition (Paper Eq. 6): Option value equals intrinsic value at maturity
if option_type == 'put':
    exercise_values = np.maximum(strike_price - bins_centers, 0)
else:
    exercise_values = np.maximum(bins_centers - strike_price, 0)

value_matrix[:, n_time_steps] = exercise_values

```

Step 3: Backward Recursion

```

for t in range(n_time_steps - 1, -1, -1): # Iterate backwards from T-1 to 0
    # Continuation value (Holding value)
    continuation_values = np.exp(-risk_free_rate * time_step) * p_matrix @
    value_matrix[:, t + 1]

    if option_style == 'eu':
        # European option: Can only be exercised at maturity
        value_matrix[:, t] = continuation_values
    else:
        # American option: Compare immediate exercise vs. holding
        value_matrix[:, t] = np.maximum(exercise_values, continuation_values)

```

Step 4: Extract Option Price

```

current_price = model_params['F_t']
current_state_index = int(np.argmin(np.abs(bins_centers - current_price))) #
Find the nearest index
option_price = value_matrix[current_state_index, 0]

```

2.3 CEV-RS Option Pricing

State Space Dimension: $2N$ (N price states $\times$ 2 regimes)

Initialization:

```

n_total_states = 2 * n_price_states
value_matrix = np.zeros((n_total_states, n_time_steps + 1))

# The terminal value for both regimes is the same (depends only on price)
value_matrix[:n_price_states, n_time_steps] = exercise_values
value_matrix[n_price_states:, n_time_steps] = exercise_values

```

Backward Recursion (2N Dimension):

```

for t in range(n_time_steps - 1, -1, -1):
    continuation_values = (
        np.exp(-risk_free_rate * time_step) * p_matrix @ value_matrix[:, t + 1]
    )

    if option_style == 'eu':
        value_matrix[:, t] = continuation_values
    else:
        # American option: Exercise decision across 2N states
        exercise_values_2n = np.concatenate([exercise_values, exercise_values])
        value_matrix[:, t] = np.maximum(exercise_values_2n, continuation_values)

```

Early Exercise Boundary Extraction (Corresponds to Paper Figure 2):

```

def find_exercise_boundary(option_values, exercise_values, continuation_values,
                           bins_centers, option_type):
    """
    Find the early exercise boundary price under a single regime.

    Put Option: The highest price where immediate exercise is better than
    holding.
    Call Option: The lowest price where immediate exercise is better than
    holding.
    """
    exercise_optimal = exercise_values > continuation_values + 1e-10

    if option_type == 'put':
        # Find the highest price for optimal exercise
        exercise_indices = np.where(exercise_optimal)[0]
        if len(exercise_indices) > 0:
            boundary_index = exercise_indices[-1]
            # Linear interpolation to improve precision
            if boundary_index < n_states - 1:
                e_diff = exercise_values[boundary_index] -
continuation_values[boundary_index]
                e_diff_next = (exercise_values[boundary_index + 1]
                               - continuation_values[boundary_index + 1])
                if e_diff != e_diff_next:
                    weight = e_diff / (e_diff - e_diff_next)
                    boundary = (bins_centers[boundary_index]
                               + weight * (bins_centers[boundary_index + 1]
                                             - bins_centers[boundary_index]))
            else:
                return bins_centers[0]

```

Initial State Selection:

```

initial_regime = model_params.get('initial_regime', 1)

if initial_regime == 1:
    current_state = price_index # Regime 1: [0, N)
else:
    current_state = n_price_states + price_index # Regime 2: [N, 2N]

option_price = value_matrix[current_state, 0]

```

Numerical Stability:

```
# Prevent division by zero for zero-sum rows
row_sums = p_matrix.sum(axis=1)
row_sums[row_sums == 0] = 1
p_matrix = p_matrix / row_sums[:, np.newaxis]
```

2.4 Numerical Stability of the Transition Probability Matrix

```
def calculate_stable_transition_matrix(q_matrix, time_step, max_norm=10):
    """
    Use sub-stepping when ||Q·dt|| is too large.

    Sub-stepping method: Divide Δt into multiple small steps, calculating
    transition probabilities iteratively.
    """
    matrix_norm = np.linalg.norm(q_matrix * time_step)

    if matrix_norm > max_norm:
        n_substeps = int(np.ceil(matrix_norm / max_norm))
        sub_time_step = time_step / n_substeps

        p_matrix = np.eye(q_matrix.shape[0])
        p_substep = expm(q_matrix * sub_time_step)
        for _ in range(n_substeps):
            p_matrix = p_matrix @ p_substep
    else:
        p_matrix = expm(q_matrix * time_step)

    # Clip and normalize
    p_matrix = np.clip(p_matrix, 0, 1)
    row_sums = p_matrix.sum(axis=1)
    p_matrix = p_matrix / row_sums[:, np.newaxis]

    return p_matrix
```

Module 3: src/parameter_estimation.py

Functionality: Calibrate model parameters from market option data.

3.1 Parameter Estimation Process

```
def estimate_parameters_from_options(option_data, model_params, option_params,
                                     ctmc_params, initial_guess, model_type,
                                     target_date=None):
    """
    Returns: optimal_params (numpy array)
    """
```

Step 1: Data Preparation

```

# Filter option data for a specific date
daily_data = option_data[option_data['Date'] == target_date]

# Lower resolution during calibration (for acceleration)
calib_ctmc = copy.deepcopy(ctmc_params)
calib_ctmc['num_bins'] = 100 # Final pricing uses 500
calib_ctmc['N_s'] = 100
calib_ctmc['N_t'] = 100 # Final pricing uses 1000

```

Step 2: Define Objective Function

```

def objective(params):
    """Minimize Log Relative Error"""
    if not validate_parameters(params, model_type):
        return 1e10 # Penalty for invalid parameters

    total_loss, n = 0.0, 0
    for _, row in daily_data.iterrows():
        try:
            # Calculate model price for each option
            m_p, mkt_p = _price_row(row, params, calib_ctmc)

            if m_p is not None and mkt_p > 0 and m_p > 0:
                # Log relative error (a more stable metric)
                total_loss += (np.log(m_p / mkt_p)) ** 2
                n += 1
        except:
            continue

    return total_loss / max(n, 1)

```

Internal Pricing Function:

```

def _price_row(row, params, ctmc_p):
    # Create local parameters for each option (preventing global template
    # pollution)
    local_model = copy.deepcopy(model_params)
    local_option = copy.deepcopy(option_params)
    local_ctmc = copy.deepcopy(ctmc_p)

    local_option['K'] = row['K']
    local_option['r_f'] = row['Rf']
    local_option['toT'] = row['TAU']
    local_model['F_t'] = row['F_t']

    # Set state space bounds based on model type and option features
    x_lower, x_upper = set_state_space_range(local_ctmc['F'], row['K'],
    local_option['option_type'],
    model_type=model_type)

    # Construct grid
    _, bins = discretize_data(local_ctmc['F'], local_ctmc['num_bins'], x_lower,
    x_upper)
    bins_centers = (bins[:-1] + bins[1:]) / 2

    # Construct Q matrix

```

```

q = construct_theoretical_q_matrix(bins_centers, local_model['F_0'], params,
model_type)

# Price the option
res = price_option(q, bins_centers, local_model, local_option, local_ctmc,
model_type)

return res[0], row['market_price'] # (Model Price, Market Price)

```

Step 3: Optimization Algorithm

```

# CEV/CEV-DEJD: Direct L-BFGS-B optimization
return opt.minimize(objective, initial_guess, method='L-BFGS-B',
bounds=bounds).x

# CEV-RS: Multi-start optimization (to avoid local optima)
def _multistart_optimize(objective, initial_guess, bounds, model_type, counter):
    n_starts = 3
    rng = np.random.default_rng(42)

    start_points = [np.array(initial_guess)]
    for _ in range(n_starts - 1):
        # Generate random start points near the initial guess
        p = initial_guess * (1.0 + rng.uniform(-0.2, 0.2, len(initial_guess)))
        start_points.append(np.clip(p, [b[0] for b in bounds], [b[1] for b in
bounds]))

    best_p, best_v = initial_guess, np.inf
    for x0 in start_points:
        try:
            res = opt.minimize(objective, x0, bounds=bounds, method='L-BFGS-B')
            if res.fun < best_v:
                best_v, best_p = res.fun, res.x
        except:
            continue

    return best_p

```

Initial Guess Setup (Paper Table 6):

```

def get_initial_guess(model_type='cev'):
    if model_type == 'cev':
        return np.array([0.2, -0.5]) # sigma, beta
    elif model_type == 'cev_dejd':
        return np.array([0.2, -0.5, 0.1, 0.5, 2.0, 2.0]) # + lambda, p, eta1,
eta2
    elif model_type == 'cev_rs':
        return np.array([0.16, 1.0, 0.16, -1.0, 1.0, 7.0]) # + sigma1, beta1,
sigma2, beta2, lambda12, lambda21

```

3.2 Calculating Pricing Performance

```
def calculate_daily_pricing_performance(daily_data, optimal_params,
                                       model_params, option_params, ctmc_params,
                                       model_type):
    """
    Compute model prices for all options on a given day using optimal
    parameters.

    Returns: DataFrame containing Date, K, TAU, Mkt_Price, Model_Price, RAE_%
    """
    results = []
    for _, row in daily_data.iterrows():
        # --- Crucial Fix: Use local_model and local_option instead of global
        # templates ---
        local_model = copy.deepcopy(model_params)
        local_option = copy.deepcopy(option_params)
        local_ctmc = copy.deepcopy(ctmc_params)

        local_option['K'] = row['K']
        local_option['r_f'] = row['Rf']
        local_option['toT'] = row['TAU']
        local_model['F_t'] = row['F_t']
        local_ctmc['dt'] = row['TAU'] / local_ctmc['N_t']

        # Restore high-resolution grid
        x_lower, x_upper = set_state_space_range(local_ctmc['F'], row['K'],
                                                local_option['option_type'],
                                                model_type=model_type)
        _, bins = discretize_data(local_ctmc['F'], local_ctmc['N_s'], x_lower,
                                x_upper)
        bins_centers = (bins[:-1] + bins[1:]) / 2

        q = construct_theoretical_q_matrix(bins_centers, local_model['F_0'],
                                          optimal_params, model_type)

        # Pass correctly updated local_model/local_option
        pricing_res = price_option(q, bins_centers, local_model, local_option,
                                local_ctmc, model_type)

        results.append({
            'Date': row['Date'], 'K': row['K'], 'TAU': round(row['TAU'], 4),
            'Mkt_Price': round(row['market_price'], 4),
            'Model_Price': round(pricing_res[0], 4),
            'RAE_%': round(abs(pricing_res[0] - row['market_price']) /
                           row['market_price'] * 100, 2)
        })

    return pd.DataFrame(results)
```

Relative Absolute Error (Paper Eq. 20):

$$RAE_i = \frac{|P_{\text{model}}(K_i, \tau_i) - P_{\text{market}}(K_i, \tau_i)|}{P_{\text{market}}(K_i, \tau_i)}$$
$$\text{Mean RAE} = \frac{1}{N} \sum_{i=1}^N RAE_i$$

3.3 Parameter Validation

```
def validate_parameters(params, model_type='cev'):  
    """Lightweight sanity check before constructing the Q matrix"""  
  
    if model_type == 'cev':  
        if len(params) != 2: return False  
        sigma, beta = params  
        if not (0 < sigma < 1): return False  
        if abs(beta) > 10: return False  
  
    elif model_type == 'cev_dejd':  
        if len(params) != 6: return False  
        sigma, beta, lam, p, eta1, eta2 = params  
        if not (0 < sigma < 1): return False  
        if abs(beta) > 10: return False  
        if not (0 < lam <= 2): return False  
        if not (0 < p < 1): return False  
        if eta1 <= 1: return False      # MGF existence condition  
        if not (0 < eta2 <= 10): return False  
  
    elif model_type == 'cev_rs':  
        if len(params) != 6: return False  
        s1, b1, s2, b2, l12, l21 = params  
        if not (0 < s1 < 1): return False  
        if not (0 < s2 < 1): return False  
        if abs(b1) > 10: return False  
        if abs(b2) > 10: return False  
        if not (0 < l12 <= 20): return False  
        if not (0 < l21 <= 20): return False  
  
    return True
```

3.4 Regime Analysis Tools (CEV-RS)

```
def calculate_stationary_distribution(lambda12, lambda21):  
    """Stationary distribution:  $\pi_1 = \lambda_{21}/(\lambda_{12}+\lambda_{21})$ ,  $\pi_2 = \lambda_{12}/(\lambda_{12}+\lambda_{21})$ """  
    total = lambda12 + lambda21  
    if total <= 0:  
        return 0.5, 0.5  
    pi1, pi2 = lambda21 / total, lambda12 / total  
    return pi1, pi2  
  
def calculate_expected_regime_duration(lambda12, lambda21):  
    """Expected duration:  $E[T_1]=1/\lambda_{12}$ ,  $E[T_2]=1/\lambda_{21}$  (years)"""  
    if lambda12 <= 0 or lambda21 <= 0:  
        return np.inf, np.inf  
    e1, e2 = 1.0 / lambda12, 1.0 / lambda21  
    return e1, e2  
  
def analyze_regime_parameters(params):  
    """Economic interpretation of CEV-RS estimates"""  
    s1, b1, s2, b2, l12, l21 = params  
  
    pi1, pi2 = calculate_stationary_distribution(l12, l21)  
    ed1, ed2 = calculate_expected_regime_duration(l12, l21)
```

```

analysis = {
    'regime1_volatility': s1,
    'regime2_volatility': s2,
    'regime1_elasticity': b1,
    'regime2_elasticity': b2,
    'regime1_leverage_type': 'inverse' if b1 > 0 else 'normal',
    'regime2_leverage_type': 'inverse' if b2 > 0 else 'normal',
    'lambda12': l12,
    'lambda21': l21,
    'stationary_prob_regime1': pi1,
    'stationary_prob_regime2': pi2,
    'expected_duration_regime1': ed1,
    'expected_duration_regime2': ed2,
    'dominant_regime': 1 if pi1 > pi2 else 2,
}

# Determine dominant leverage type
analysis['dominant_leverage_type'] = (
    analysis['regime1_leverage_type'] if pi1 > pi2
    else analysis['regime2_leverage_type']
)

return analysis

```

Module 4: utility/data_processor.py

Functionality: Data processing, state space setup, and data analysis.

4.1 Data Discretization

```

def discretize_data(data, num_bins, x_lower, x_upper):
    """
    Discretize continuous data into a specified number of grid points.

    Parameters:
        data: Input data array
        num_bins: Number of bins
        x_lower: Data lower bound
        x_upper: Data upper bound
    Returns: (indices, bins) - State index sequence and grid boundaries
    """
    # Expand boundaries to ensure all data falls within the range
    x_min = x_lower - 1
    x_max = x_upper + 1

    # Create equally spaced bins
    bins = np.linspace(x_min, x_max, num_bins + 1)

    # Distribute data into bins
    indices = np.digitize(data, bins) - 1

    # Ensure indices are within the valid range
    indices = np.clip(indices, 0, num_bins - 1)

```

```
return indices, bins
```

4.2 Setting the State Space Range

```
def set_state_space_range(futures_data, strike_price, option_type,
model_type='cev',
                        delta_upper=None, delta_lower=None):
    """
    Set discretization bounds based on model type and option features.

    Parameters:
        futures_data: Futures price data
        strike_price: Strike price
        option_type: Option type
        model_type: Model type
        delta_upper/delta_lower: Upper/lower bound expansion ratio (Optional;
otherwise model defaults are used)
    Returns: (x_lower, x_upper) - Lower and upper bounds of the state space
    """
    max_price = np.max(futures_data)
    min_price = np.min(futures_data)

    # Select expansion ratio based on model type
    if delta_upper is None or delta_lower is None:
        if model_type == 'cev':
            delta_upper, delta_lower = 0.05, 0.02
        elif model_type == 'cev_dejd':
            delta_upper, delta_lower = 0.15, 0.10 # Jumps require a wider range
        elif model_type == 'cev_rs':
            delta_upper, delta_lower = 0.10, 0.08 # Regime switching requires a
moderate range

    # Adjust boundaries based on option type and strike price
    if option_type == 'call':
        if strike_price > max_price:
            x_upper = strike_price * (1 + delta_upper)
            x_lower = min_price * (1 - delta_lower)
        else:
            x_upper = max_price * (1 + delta_upper)
            x_lower = min_price * (1 - delta_lower)
    else: # put
        if strike_price < min_price:
            x_upper = max_price * (1 + delta_upper)
            x_lower = strike_price * (1 - delta_lower)
        else:
            x_upper = max_price * (1 + delta_upper)
            x_lower = min_price * (1 - delta_lower)

    # CEV-DEJD extra adjustment: ensure a minimum range
    if model_type == 'cev_dejd':
        price_range = x_upper - x_lower
        min_range = max_price * 0.5 # Use 50% of the max price as the minimum
range
        if price_range < min_range:
            center = (x_upper + x_lower) / 2
            x_upper = center + min_range / 2
            x_lower = center - min_range / 2
```

```

        # Ensure x_lower is positive
        if x_lower <= 0:
            x_lower = max_price * 0.1

    # CEV-RS extra adjustment: ensure sufficient range
    if model_type == 'cev_rs':
        price_range = x_upper - x_lower
        min_range = max_price * 0.4 # Use 40% of the max price as the minimum
range
        if price_range < min_range:
            center = (x_upper + x_lower) / 2
            x_upper = center + min_range / 2
            x_lower = center - min_range / 2
        # Ensure x_lower is positive
        x_lower = max(x_lower, max_price * 0.05) # At least 5% of max price

    return x_lower, x_upper

```

Model-Specific Needs:

- **CEV-DEJD:** Needs a wider range to accommodate jumps.
- **CEV-RS:** Needs sufficient range to accommodate volatility shifts driven by regimes.

4.3 Filtering Option Data

```

def filter_option_data(option_data, filter_criteria):
    """
    Filter option data based on liquidity criteria.

    Parameters:
        option_data: Raw option data
        filter_criteria: Dictionary of filtering conditions
    Returns: Filtered option data
    """
    filtered_data = option_data.copy()

    # 1. Time to maturity filter (15 days <  $\tau$  < 180 days)
    filtered_data = filtered_data[
        (filtered_data["TAU"] >= filter_criteria['tau_min']) &
        (filtered_data["TAU"] <= filter_criteria['tau_max'])
    ]

    # 2. Moneyness filter (0.81 < Moneyness < 1.02)
    filtered_data = filtered_data[
        (filtered_data["MONEYNESSESS"] > filter_criteria['moneyness_min']) &
        (filtered_data["MONEYNESSESS"] <= filter_criteria['moneyness_max'])
    ]

    # 3. Volume filter (Volume > 100)
    filtered_data = filtered_data[
        filtered_data["volume"] > filter_criteria['volume_min']
    ]

    # 4. Price filter (Market Price > 0.50)
    filtered_data = filtered_data[
        filtered_data["market_price"] > filter_criteria['price_min']
    ]

```

```

# Extract required columns
result_columns = ['Date', 'K', 'TAU', 'Rf', 'market_price', 'MONEYNESS',
                  'F_t']

return filtered_data[result_columns]

```

Liquidity Filtering Logic:

- **Time to maturity:** Short-term options (15-180 days) have good liquidity.
- **Moneyness:** Deep in-the-money (0.81-1.02) avoids pricing difficulties with deep out-of-the-money options.
- **Volume:** High volume (>100) ensures reliable pricing.
- **Minimum price:** Avoids extremely low prices from deep out-of-the-money options.

4.4 Data Analysis Tools

Jump Characteristics Analysis (CEV-DEJD):

```

def analyze_jump_characteristics(futures_data, threshold_pct=0.02):
    """
    Analyze potential jump characteristics in futures data
    to help set appropriate parameters for the CEV-DEJD model.

    Parameters:
        futures_data: Array of futures prices
        threshold_pct: Threshold percentage for identifying jumps (default 2σ)
    Returns: Dictionary containing jump statistics
    """
    returns = np.diff(np.log(futures_data))
    threshold = np.std(returns) * 2 # 2 standard deviations

    # Identify potential jumps
    large_moves = np.abs(returns) > threshold
    positive_jumps = returns > threshold
    negative_jumps = returns < -threshold

    jump_stats = {
        'total_observations': len(returns),
        'potential_jumps': np.sum(large_moves),
        'jump_frequency': np.sum(large_moves) / len(returns),
        'positive_jumps': np.sum(positive_jumps),
        'negative_jumps': np.sum(negative_jumps),
        'jump_asymmetry': np.sum(positive_jumps) / max(np.sum(large_moves), 1),
        'avg_positive_jump': np.mean(returns[positive_jumps]),
        'avg_negative_jump': np.mean(returns[negative_jumps]),
        'max_positive_jump': np.max(returns),
        'max_negative_jump': np.min(returns),
    }

    return jump_stats

```

Regime Characteristics Analysis (CEV-RS):

```

def analyze_regime_characteristics(futures_data, window_size=20):
    """

```

Analyze potential regime-switching traits in futures data to help set appropriate parameters for the CEV-RS model.

Parameters:

futures_data: Array of futures prices

window_size: Rolling window size for calculating volatility

Returns: Dictionary containing regime statistics

"""

```
returns = np.diff(np.log(futures_data))
```

```
# Calculate rolling volatility (annualized)
```

```
rolling_vol = pd.Series(returns).rolling(window=window_size).std() *  
np.sqrt(252)  
rolling_vol = rolling_vol.dropna().values
```

```
# Identify high and low volatility regimes
```

```
vol_median = np.median(rolling_vol)  
high_vol_regime = rolling_vol > vol_median  
low_vol_regime = rolling_vol <= vol_median
```

```
# Calculate regime statistics
```

```
regime_stats = {  
    'total_periods': len(rolling_vol),  
    'high_vol_periods': np.sum(high_vol_regime),  
    'low_vol_periods': np.sum(low_vol_regime),  
    'high_vol_proportion': np.sum(high_vol_regime) / len(rolling_vol),  
    'avg_high_vol': np.mean(rolling_vol[high_vol_regime]),  
    'avg_low_vol': np.mean(rolling_vol[low_vol_regime]),  
    'vol_ratio': np.mean(rolling_vol[high_vol_regime]) /  
np.mean(rolling_vol[low_vol_regime]),  
}
```

```
# Estimate transition frequency (annualized)
```

```
transitions_to_high = 0
```

```
transitions_to_low = 0
```

```
for i in range(1, len(high_vol_regime)):
```

```
    if high_vol_regime[i] and not high_vol_regime[i - 1]:  
        transitions_to_high += 1
```

```
    elif not high_vol_regime[i] and high_vol_regime[i - 1]:  
        transitions_to_low += 1
```

```
periods_per_year = 252 / window_size
```

```
regime_stats['estimated_lambda12'] = transitions_to_high /  
max(regime_stats['low_vol_periods'], 1) * periods_per_year  
regime_stats['estimated_lambda21'] = transitions_to_low /  
max(regime_stats['high_vol_periods'], 1) * periods_per_year
```

```
return regime_stats
```

Leverage Effect Analysis:

```
def analyze_leverage_effect(futures_data, window_size=20):
```

```
    """
```

```
    Analyze the leverage effect in futures data.
```

```
    Positive correlation = Inverse leverage, Negative correlation = Normal  
    leverage.
```

```

Parameters:
    futures_data: Array of futures prices
    window_size: Rolling window size
Returns: Dictionary containing leverage analysis
"""
returns = np.diff(np.log(futures_data))

# Calculate rolling volatility
rolling_vol = pd.Series(returns).rolling(window=window_size).std()
vol_changes = rolling_vol.diff().dropna().values

# Align returns with volatility changes
aligned_returns = returns[window_size:]
if len(vol_changes) > len(aligned_returns):
    aligned_returns = aligned_returns[:len(vol_changes)]
elif len(vol_changes) < len(aligned_returns):
    vol_changes = vol_changes[:len(aligned_returns)]

# Calculate correlation coefficient
valid_mask = ~(np.isnan(aligned_returns) | np.isnan(vol_changes))
correlation = np.corrcoef(aligned_returns[valid_mask],
vol_changes[valid_mask])[0, 1]

leverage_analysis = {
    'correlation': correlation,
    'leverage_type': 'inverse' if correlation > 0 else 'normal',
    'strength': abs(correlation),
    'interpretation': 'strong' if abs(correlation) > 0.3 else 'Moderate' if
abs(correlation) > 0.1 else 'weak'
}

return leverage_analysis

```

4.5 Parameter Suggestion Tool

```

def suggest_state_space_params(futures_data, model_type='cev'):
    """
    Suggest optimal state space parameters based on data traits.

    Returns: Dictionary containing suggested parameters
    """
    if model_type == 'cev_dejd':
        jump_stats = analyze_jump_characteristics(futures_data)
        # Adjust parameters based on jump characteristics
        if jump_stats['jump_frequency'] > 0.05: # High jump frequency
            delta_upper, delta_lower, num_bins = 0.20, 0.15, 600
        elif jump_stats['jump_frequency'] > 0.02: # Moderate jump frequency
            delta_upper, delta_lower, num_bins = 0.15, 0.10, 550
        else: # Low jump frequency
            delta_upper, delta_lower, num_bins = 0.10, 0.08, 500

    elif model_type == 'cev_rs':
        regime_stats = analyze_regime_characteristics(futures_data)
        leverage_analysis = analyze_leverage_effect(futures_data)
        # Adjust parameters based on regime characteristics
        if regime_stats is not None:
            vol_ratio = regime_stats.get('vol_ratio', 1.5)

```

```

        if vol_ratio > 2.0: # High volatility contrast
            delta_upper, delta_lower, num_bins = 0.15, 0.12, 550
        elif vol_ratio > 1.5: # Moderate contrast
            delta_upper, delta_lower, num_bins = 0.10, 0.08, 500
        else: # Low contrast
            delta_upper, delta_lower, num_bins = 0.08, 0.06, 450
    else:
        # Default CEV suggestions
        delta_upper, delta_lower, num_bins = 0.05, 0.02, 500

    return suggestions

```

Module 5: utility/file_utils.py

Functionality: File I/O operations supporting the saving of multiple data types.

5.1 Read Data

```

def read_futures_data(filepath):
    """Read futures price data"""
    futures_info = pd.read_csv(filepath)
    futures_price_data = futures_info[['Date', 'close']]
    return futures_price_data

def read_option_data(filepath):
    """Read option market data"""
    option_info = pd.read_csv(filepath)
    return option_info

```

5.2 Save Results

```

def save_results(data, filename, result_dir='result', model_type='cev',
timestamp=True):
    """
    Save the result data to the specified directory with a model type
    identifier.

    Supported formats: DataFrame, numpy array, dict
    """
    # Ensure directory exists
    if not os.path.exists(result_dir):
        os.makedirs(result_dir)

    # Construct filename: base_name_{model_type}_{timestamp}.csv
    base_name, ext = os.path.splitext(filename)
    if not ext:
        ext = '.csv'

    if timestamp:
        timestamp_str = datetime.now().strftime("%Y%m%d_%H%M%S")
        final_filename = f"{base_name}_{model_type}_{timestamp_str}{ext}"
    else:
        final_filename = f"{base_name}_{model_type}{ext}"

```



```

filepath = os.path.join(result_dir, final_filename)

# Save based on type
if isinstance(data, pd.DataFrame):
    data.to_csv(filepath, index=False)
elif isinstance(data, np.ndarray):
    pd.DataFrame(data).to_csv(filepath, index=False)
elif isinstance(data, dict):
    pd.DataFrame(data).to_csv(filepath, index=False)
else:
    pd.DataFrame(data).to_csv(filepath, index=False)

```

5.3 Backup and Cleanup

```

def backup_results(result_dir='result', backup_dir='backup'):
    """
    Create a backup of existing results before running anew.
    """
    # Create timestamped backup directory
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    specific_backup_dir = os.path.join(backup_dir, f"backup_{timestamp}")

    # Copy all files
    for filename in os.listdir(result_dir):
        src_path = os.path.join(result_dir, filename)
        dst_path = os.path.join(specific_backup_dir, filename)
        if os.path.isfile(src_path):
            shutil.copy2(src_path, dst_path)

def clean_old_results(result_dir='result', days_old=30):
    """
    Clean up old result files.
    """
    cutoff_time = current_time - (days_old * 24 * 60 * 60)

    removed_count = 0
    for filename in os.listdir(result_dir):
        filepath = os.path.join(result_dir, filename)
        if os.path.isfile(filepath):
            file_time = os.path.getmtime(filepath)
            if file_time < cutoff_time:
                os.remove(filepath)
                removed_count += 1

```

5.4 Load Historical Results

```

def load_previous_results(model_type, result_dir='result'):
    """
    Load previous results for a specified model type.

    Returns: Dictionary containing parameters, pricing, and early exercise
    boundaries (for CEV-RS)
    """
    results = {}

    # Load parameter results

```

```

param_file = os.path.join(result_dir,
f"estimated_parameters_{model_type}.csv")
if os.path.exists(param_file):
    results['parameters'] = pd.read_csv(param_file)

# Load pricing results
pricing_file = os.path.join(result_dir,
f"option_pricing_results_{model_type}.csv")
if os.path.exists(pricing_file):
    results['pricing'] = pd.read_csv(pricing_file)

# CEV-RS: Load early exercise boundaries
if model_type == 'cev_rs':
    boundary_files = [f for f in os.listdir(result_dir) if
f.startswith('exercise_boundary_cev_rs')]
    if boundary_files:
        boundary_files.sort(reverse=True)
        boundary_file = os.path.join(result_dir, boundary_files[0])
        results['exercise_boundaries'] = pd.read_csv(boundary_file)

return results if results else None

```

Operation Process

```

main()
├─ 1. Initialize the log system
|   └─ logger.remove()
|   └─ logger.add(sys.stderr, level="INFO")
|   └─ logger.add(log_file_path, level="INFO", format=LOG_CONFIG['format'])
|
├─ 2. Create directory structure
|   └─ create_directories([FILE_PATHS['log_dir'], FILE_PATHS['result_dir']])
|
├─ 3. Read data
|   └─ f_data = read_futures_data(FILE_PATHS['futures_data'])
|       └─ DataFrame: [Date, close]
|   └─ o_data_raw = read_option_data(FILE_PATHS['options_data'])
|       └─ DataFrame: [Date, K, TAU, Rf, market_price, MONEYNESS, F_t, volume]
|
├─ 4. Data Preprocessing
|   └─ o_data = filter_option_data(o_data_raw, OPTION_FILTER_CRITERIA)
|       └─ DataFrame: [Date, K, TAU, Rf, market_price, MONEYNESS, F_t]
|           (Selected: 15天≤T≤180天, 0.81<Moneyness≤1.02, Volume>100, Price>0.50)
|
├─ 5. Initialize Parameters
|   └─ m_params = copy.deepcopy(DEFAULT_MODEL_PARAMS)
|       └─ m_params['F_0'] = f_data['close'][0]
|   └─ c_params = copy.deepcopy(DEFAULT_CTMC_PARAMS)
|       └─ c_params['F'] = np.asarray(f_data['close'][:])
|       └─ c_params['N'] = len(c_params['F']) - 1
|
├─ 6. Create a directory structure and calibrate and price for each target
date.
|   └─ for date in target_dates: # [20240603, 20240606]
|       |

```

```

|         |— 6.1 Parameter Estimation
|         |   |— opt_p = estimate_parameters_from_options(o_data, m_params,
DEFAULT_OPTION_PARAMS,
|         |                                     c_params,
get_initial_guess(model_type), model_type, date)
|         |   |— 返回: numpy array(Optimal Parameters)
|         |
|         |— 6.2 Generate pricing reports
|         |   |— day_data = o_data[o_data['Date'] == date]
|         |   |— perf_df = calculate_daily_pricing_performance(
|         |               day_data, opt_p, m_params, DEFAULT_OPTION_PARAMS,
c_params, model_type)
|         |
|         |— 6.3 Log output
|         |   |— logger.info(f"\n[ Pricing Report for {date}
]\n{perf_df.to_string(index=False)}")
|         |   |— names = get_parameter_names(model_type)
|         |   |— optimal_params_dict = dict(zip(names, np.round(opt_p, 4)))
|         |   |— logger.info(f"Optimal Parameters: {optimal_params_dict}\n")
|         |
|         |— all_results.append(perf_df)
|
|— 7. Summarize and save the results
|   |— final = pd.concat(all_results)
|   |— summary_text = (
|     f"\n{'=' * 50}\n"
|     f"FINAL BATCH SUMMARY ({model_type.upper()})\n"
|     f"Total Options: {len(final)}\n"
|     f"Global Mean RAE: {final['RAE_%'].mean():.2f}%\n"
|     f"{'=' * 50}"
|   )
|   |— logger.info(summary_text)
|   |— save_results(final, "batch_pricing_report", FILE_PATHS['result_dir'],
model_type, timestamp=True)
|     |— Save as: batch_pricing_report_cev_20260319_HHMMSS.csv

```